

Homework 3: Plan-and-Execute Data Analysis Agent

ECE 157C / ECE 272C — University of California, Santa Barbara Due: May 15, 2026

1. Dataset Description

Source: Los Angeles Open Data Portal — [LA Parking Citations](#) (sampled 20,000 rows via Socrata API, cleaned to 19,966 usable rows)

Rows / Columns: 19,966 rows × 16 columns

Key Columns:

Column	Type	Description
issue_date	string	Date the citation was issued (YYYY-MM-DD)
issue_time	float	Time as a 4-digit number (e.g., 1430 = 2:30 PM)
rpstateplate	string	State that issued the vehicle's license plate
make	string	Vehicle make abbreviation (e.g., TOYT, HOND, FORD)
body_style	string	Vehicle body style code (PA = passenger, PU = pickup, VN = van, etc.)
color	string	Vehicle color code
location	string	Street address where the citation was issued
violation_code	string	Numeric/alpha code identifying the violation
violation_description	string	Human-readable description of the violation
fine_amount	float	Dollar amount of the fine
latitude / longitude	float	GPS coordinates of the citation location

Why this dataset requires multi-step analysis: Single-step queries cannot answer meaningful questions about this data. For example, finding the vehicle make with the highest average fine *among makes with more than 100 citations* requires aggregation, filtering on the aggregate, and sorting — three distinct operations. Similarly, deriving an "issue hour" from the raw `issue_time` field (e.g., 1430 → 14) requires a column derivation before any grouping can happen. The mix of numeric fines, categorical makes/violations, encoded timestamps, and geographic data naturally produces questions that span multiple operator types.

2. Question Design

Basic Questions (5)

#	Question	Why multi-step	Expected operations	Classification rationale
B1	What are the top 5 vehicle makes by number of citations received?	Aggregation alone isn't enough — results must be ranked and truncated	<code>groupandaggregate</code> → <code>sortrows</code> → <code>limitrows</code>	3 sequential ops, no filtering or derivation needed
B2	What is the average fine amount for citations issued to out-of-state (non-CA) vehicles?	Must first narrow the dataset before computing the statistic	<code>filterrows (state ≠ CA)</code> → <code>groupand_aggregate</code> (mean fine)	Simple filter + aggregate, single condition
B3	What are the top 3 most expensive violation types by average fine amount?	Must aggregate per group then rank	<code>groupandaggregate</code> (mean fine by violation) → <code>sortrows (desc)</code> → <code>limitrows (3)</code>	Aggregate + rank + truncate, no derivation
B4	How many citations were issued per body style, ordered from most to least?	Requires counting per group then ordering	<code>groupandaggregate</code> (count by <code>bodystyle</code>) → <code>sortrows (desc)</code>	Two ops; no filtering needed
B5	What are the distinct violation descriptions for citations with a fine of exactly \$73?	Must filter then deduplicate	<code>filterrows (fine = 73)</code> → <code>selectcolumns ([violationdescription])</code> → <code>distinctrows</code>	Filter + project + dedup

Intermediate Questions (5)

#	Question	Why multi-step	Expected operations	Classification rationale
I1	Among Toyota (TOYT) vehicles, what are the top 3 violation types?	Subset before aggregating; result must be ranked	<code>filterrows (make = TOYT)</code> → <code>groupandaggregate (count by violation)</code> → <code>sortrows (desc)</code> → <code>limit_rows (3)</code>	Adds a pre-filter step before the basic aggregate+rank pattern
I2	Which violation types generated total fine revenue above \$50,000?	Aggregate first, then filter on the aggregate result	<code>groupandaggregate (count + sum fine by violation)</code> → <code>filterrows (totalfine > 50000)</code> → <code>sort_rows (desc)</code>	Post-aggregation filter — requires thinking in two phases
I3	For citations issued during morning hours (<code>issue_time < 1200</code>), what is the most common violation?	Time-based filter narrows the scope; then requires aggregate + rank	<code>filterrows (issuetime < 1200)</code> → <code>groupandaggregate (count by violation)</code> → <code>sortrows (desc)</code> → <code>limitrows (1)</code>	Domain-specific numeric condition combined with aggregate + rank
I4	What are the top 5 locations with the most RED ZONE violations?	Must isolate one violation type then aggregate by a different column	<code>filterrows (violation = RED_ZONE)</code> → <code>groupandaggregate (count by location)</code> →	Cross-column filter → aggregate → rank

#	Question	Why multi-step	Expected operations	Classification rationale
			<i>sortrows</i> (desc) → <i>limit_rows</i> (5)	
I5	Among passenger vehicles (body_style = PA), which vehicle makes have more than 20 citations with fines above \$100?	Two independent filter conditions, then aggregate, then post-aggregate filter	<i>filterrows</i> (bodystyle = PA) → <i>filterrows</i> (fine > 100) → <i>groupandaggregate</i> (count by make) → <i>filterrows</i> (count > 20) → <i>sort_rows</i> (desc)	Two pre-filters + aggregate + post-aggregate filter = 5 steps

Advanced Questions (5)

#	Question	Why multi-step	Expected operations	Classification rationale
A1	Derive the issue hour from issue_time (floor divide by 100), then find the top 3 hours by total fine revenue.	Raw data doesn't have an hour column — must derive it before any aggregation	<i>derivecolumns</i> (hour = <i>issuetime</i> // 100) → <i>groupandaggregate</i> (sum fine by hour) → <i>sortrows</i> (desc) → <i>limitrows</i> (3)	Requires column derivation before aggregation; 4 steps
A2	Among vehicle makes with more than 100 citations, which 5 have the highest average fine amount?	Must aggregate, post-filter on count, then re-rank on a different metric	<i>groupandaggregate</i> (count + mean fine by make) → <i>filterrows</i> (count > 100) → <i>sortrows</i> (mean fine desc) → <i>limit_rows</i> (5)	Post-aggregation filter on count while ranking on a different column
A3	For each violation type, compute the ratio of total fine revenue to citation count. Which violations have a ratio above \$80, and how are they ranked?	Requires aggregation followed by a derived column, then filter on that derived column	<i>groupandaggregate</i> (count + sum fine by violation) → <i>derivecolumns</i> (ratio = <i>totalfine</i> / count) → <i>filterrows</i> (ratio > 80) → <i>sortrows</i> (ratio desc)	Post-aggregate derivation + filter on derived column
A4	Among CA-registered vehicles cited for street cleaning (NO PARK/STREET CLEAN), which 3 vehicle makes have the highest total fines among makes with more than 50 such citations?	Two pre-filters on different columns, aggregate, post-aggregate filter, rank	<i>filterrows</i> (state = CA) → <i>filterrows</i> (violation = NO PARK/STREET CLEAN) → <i>groupandaggregate</i> (count + sum fine by make) → <i>filterrows</i> (count > 50) → <i>sortrows</i> (sum fine desc) → <i>limit_rows</i> (3)	Longest chain of pre-filters before aggregate + post-aggregate filter; 6 steps
A5	Derive the issue hour, keep only afternoon citations (hour 12–18), then find the top 3 vehicle makes by total fine revenue among makes with more than 50 afternoon citations.	Derivation → time-range filter → aggregate → post-aggregate filter → rank → truncate	<i>derivecolumns</i> (hour) → <i>filterrows</i> (hour ≥ 12) → <i>filterrows</i> (hour ≤ 18) → <i>groupandaggregate</i> (count + sum fine by make) → <i>filterrows</i> (count > 50) →	Combines derivation, multi-condition time filter, aggregate, post-aggregate filter, and rank; 7 steps

#	Question	Why multi-step	Expected operations	Classification rationale
			<code>sortrows (sum.fine desc) → limitrows (3)</code>	

3. System Design

3.1 Planner

The planner (`planner.py`) takes a natural language question and a `pd.DataFrame` and returns a structured JSON plan.

Prompt structure: - A static system prompt contains the full operator reference (all 7 operators with exact JSON format) and strict rules: no natural language in output, all vague terms must be resolved to concrete values (e.g., "afternoon" → hours 12–18), no Python code, return only valid JSON. - The user message contains the dataset schema (column name, dtype, 3 sample values, total row count) followed by the question.

Schema injection: `build_schema_description()` iterates over all columns and formats each as `column_name (dtype): e.g. [sample values]`. This gives the LLM enough context to use the correct column names and understand value ranges without hallucinating column names.

Output validation: The raw response is stripped of any accidental markdown fences, then parsed with `json.loads()`. If parsing fails, an exception propagates to the caller.

Model: GPT-4o at temperature 0 for deterministic, reproducible plans.

3.2 Executor

The executor (`executor.py`) takes a plan dict and a `pd.DataFrame` and runs each step in order.

Dispatch: A static `DISPATCH` dictionary maps each `op` string to a lambda that unpacks the step's parameters and calls the matching operator function. Unknown operators raise a `ValueError` immediately with a clear message.

Intermediate results: The current table is stored in a single `current` variable. Each step overwrites it with the operator's output — no intermediate tables are retained in memory beyond what's needed.

Execution trace: Before and after each step, `len(current)` is recorded. The trace is a list of `{step, operation, input_rows, output_rows}` dicts returned alongside the final result. This is used by `agent.py` to display the step table in the report.

Return value: `execute(plan, df)` returns `(result_df, trace)` — the final table and the full trace.

3.3 Operators

All operators live in `operators.py`. Each takes a `pd.DataFrame` plus structured parameters and returns a new `pd.DataFrame` — no side effects, no natural language.

Operator	Parameters	Behavior
<code>derive_columns</code>	<code>derive</code> : list of <code>{new_column, type, operation, left, right}</code>	Adds computed columns via arithmetic (add, subtract, multiply, divide, floor_divide). Operands are column references or literals.
<code>filter_rows</code>	<code>conditions</code> : list of <code>{column, operator, value}</code>	Keeps rows satisfying all conditions (AND). Operators: <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>==</code> , <code>!=</code> .

Operator	Parameters	Behavior
<code>group_and_aggregate</code>	<code>group_by</code> : list of columns; <code>metrics</code> : list of {function, column, as}	Groups and computes mean, sum, count, min, or max per group. Multiple metrics per call are supported.
<code>sort_rows</code>	<code>sort_by</code> : list of {column, direction}	Sorts by one or more columns; direction is "asc" or "desc".
<code>limit_rows</code>	<code>k</code> : int	Returns the first k rows.
<code>select_columns</code>	<code>columns</code> : list of str	Drops all columns not in the list.
<code>distinct_rows</code>	<code>columns</code> : list of str (optional)	Removes duplicate rows, optionally scoped to a subset of columns.

3.4 Intermediate Result Passing

The executor holds a single `current` variable that starts as a copy of the original dataset. After each step, `current` is replaced with the operator's output. This means each operator always receives a clean, fully-evaluated DataFrame — there is no lazy evaluation or query graph. The final value of `current` after all steps is the result.

`agent.py` wires the three modules together: it loads the CSV, calls `generate_plan()`, passes the plan and DataFrame to `execute()`, then sends the result table to `generate_answer()` which produces a natural language response grounded only in the computed rows.

4. Execution Analysis

Question 1: B1 — Top 5 vehicle makes by citation count (Basic)

Natural language question: What are the top 5 vehicle makes by number of citations received?

Generated plan:

```
{
  "steps": [
    {
      "op": "group_and_aggregate",
      "group_by": ["make"],
      "metrics": [{"function": "count", "column": "ticket_number", "as": "citation_count"}]
    },
    {"op": "sort_rows", "sort_by": [{"column": "citation_count", "direction": "desc"}]},
    {"op": "limit_rows", "k": 5},
    {"op": "select_columns", "columns": ["make", "citation_count"]}
  ]
}
```

Execution trace:

Step	Operation	Input Rows	Output Rows
1	groupandaggregate	19,966	99
2	sort_rows	99	99
3	limit_rows	99	5
4	select_columns	5	5

Final answer: TOYT (3,460), HOND (2,269), FORD (1,768), CHEV (1,440), NISS (1,406).

Analysis: The plan is correct. Step 1 collapses 19,966 citation rows into 99 unique makes, each with a count. Step 2 reorders those 99 rows without reducing them. Step 3 truncates to the top 5. Step 4 drops irrelevant columns for a clean result. Each step has exactly one responsibility and the row counts confirm the expected behavior — aggregation is the only step that reduces rows here.

Question 2: I2 — Violation types with total revenue above \$50,000 (Intermediate)

Natural language question: Which violation types generated total fine revenue above \$50,000?

Generated plan:

```
{
  "steps": [
    {
      "op": "group_and_aggregate",
      "group_by": ["violation_description"],
      "metrics": [{"function": "sum", "column": "fine_amount", "as": "total_fine_revenue"}]
    },
    {
      "op": "filter_rows",
      "conditions": [{"column": "total_fine_revenue", "operator": ">", "value": 50000}]
    },
    {
      "op": "select_columns", "columns": ["violation_description"]
    }
  ]
}
```

Execution trace:

Step	Operation	Input Rows	Output Rows
1	groupandaggregate	19,966	90
2	filter_rows	90	6
3	select_columns	6	6

Final answer: METER EXP., NO PARK/STREET CLEAN, NO PARKING, NO STOPPING/ANTI-GRI, PREFERENTIAL PARKING, RED ZONE.

Analysis: This question demonstrates a post-aggregation filter — a two-phase pattern where the threshold is applied to a derived aggregate, not to the raw data. Step 1 computes total revenue per violation type (90 unique types). Step 2 filters on the computed `total_fine_revenue` column, which only exists after Step 1 — this is why order matters. Step 3 cleans up the output. The plan correctly separates the aggregation phase from the filtering phase.

Question 3: A3 — Fine-to-citation ratio with post-aggregate derivation (Advanced)

Natural language question: For each violation type, compute the ratio of total fine revenue to citation count. Which violations have a ratio above \$80, ranked by ratio descending?

Generated plan:

```
{
  "steps": [
    {
      "op": "group_and_aggregate",
      "group_by": ["violation_code"],
      "metrics": [
        {"function": "sum", "column": "fine_amount", "as": "total_fine_revenue"},
        {"function": "count", "column": "ticket_number", "as": "citation_count"}
      ]
    }
  ],
}
```

```

    {
      "op": "derive_columns",
      "derive": [{
        "new_column": "fine_to_citation_ratio",
        "type": "arithmetic", "operation": "divide",
        "left": {"type": "column", "value": "total_fine_revenue"},
        "right": {"type": "column", "value": "citation_count"}
      }]
    },
    {
      "op": "filter_rows",
      "conditions": [{"column": "fine_to_citation_ratio", "operator": ">", "value": 80}]
    },
    {"op": "sort_rows", "sort_by": [{"column": "fine_to_citation_ratio", "direction": "desc"}]},
    {"op": "select_columns", "columns": ["violation_code", "fine_to_citation_ratio"]}
  ]
}

```

Execution trace:

Step	Operation	Input Rows	Output Rows
1	groupandaggregate	19,966	95
2	derive_columns	95	95
3	filter_rows	95	30
4	sort_rows	30	30
5	select_columns	30	30

Final answer: 30 violation codes have a ratio above \$80. The highest ratio (363.0) belongs to 13 codes including 570, 569, 22500L, and 22507.8A (all \$363 fixed-fine violations). The lowest qualifying ratio is 93.0.

Analysis: This is the most structurally complex plan — it chains aggregate → derive → filter → sort. The key insight is that `fine_to_citation_ratio` does not exist in the original data; it can only be computed after Step 1 creates both `total_fine_revenue` and `citation_count`. Step 2 derives it using column-to-column division. Step 3 then filters on a column that was itself derived from aggregated columns — a three-level chain. This plan would be impossible to express in a single SQL-like query without a subquery or CTE, but the step-by-step execution model handles it naturally.

5. Failure Analysis

Failure Case 1

Question: What is the average fine amount for citations issued to out-of-state (non-CA) vehicles?

Generated plan:

```

{
  "steps": [
    {
      "op": "filter_rows",
      "conditions": [{"column": "rp_state_plate", "operator": "!=", "value": "CA"}]
    },
    {
      "op": "group_and_aggregate",
      "group_by": [],
      "metrics": [{"function": "mean", "column": "fine_amount", "as": "average_fine_amount"}]
    }
  ]
}

```

```
]
}
```

What went wrong: The planner correctly reasoned that this question needs a global aggregate (no grouping key), and emitted `"group_by": []`. However, the initial `group_and_aggregate` operator passed the empty list directly to `pandas.DataFrame.groupby()`, which raises `ValueError: No group keys passed!`

Error type: - [] Planning error - [x] Operator limitation - [] Execution issue

Fix / improvement: Added a branch in `group_and_aggregate` for the `group_by == []` case. When no grouping keys are provided, the operator computes each aggregate function directly on the full column (e.g. `df[col].mean()`) and returns a single-row DataFrame. The plan was correct — only the operator needed to handle the edge case.

Failure Case 2

Question: What is the total revenue from citations issued to luxury vehicles like BMW, Mercedes, and Lexus combined?

Generated plan:

```
{
  "steps": [
    {
      "op": "filter_rows",
      "conditions": [
        {"column": "make", "operator": "==", "value": "BMW"},
        {"column": "make", "operator": "==", "value": "MERZ"},
        {"column": "make", "operator": "==", "value": "LEXS"}
      ]
    },
    {
      "op": "group_and_aggregate",
      "group_by": [],
      "metrics": [{"function": "sum", "column": "fine_amount", "as": "total_revenue"}]
    }
  ]
}
```

What went wrong: The planner correctly resolved "Mercedes" → "MERZ" and "Lexus" → "LEXS" from the schema samples, but placed all three make conditions in the same `filter_rows` step. Because `filter_rows` ANDs all conditions, a row must simultaneously satisfy `make == "BMW" AND make == "MERZ" AND make == "LEXS"` — which is impossible. The executor ran without error and returned a total of `0.0`, producing a silent wrong answer.

Error type: - [x] Planning error - [x] Operator limitation - [] Execution issue

Fix / improvement: The root cause is that `filter_rows` has no OR logic. Two fixes are possible:

1. Add an `operator_type` field to `filter_rows` — e.g. `"logic": "or"` to switch from AND to OR. This is the minimal change.
2. Add an `in` operator to `filter_rows` conditions — `{"column": "make", "operator": "in", "value": ["BMW", "MERZ", "LEXS"]}`. This is more expressive and maps cleanly to `pandas.Series.isin()`.

Option 2 was added to `operators.py` as an additional operator capability after this failure was observed.

6. Comparison with Previous Agents

Differences Between Agent Designs

Aspect	HW1 (Code-Generation)	HW2 (Interactive/ Memory)	HW3 (Plan-and-Execute)
Reasoning mechanism	LLM writes arbitrary Python to answer each question	LLM reasons interactively; memory persists context across turns	LLM generates a structured JSON plan; no code produced
Execution model	Python interpreter runs LLM-generated code directly	LangGraph nodes run code and update state	Deterministic operator pipeline; no LLM involvement at execution time
Reproducibility	Low — same question can produce different code on different runs	Medium — memory state influences outputs	High — same plan always produces the same result
Debuggability	Hard — must read and reason about generated code	Medium — can inspect conversation history and state	Easy — execution trace shows row counts at every step; operators are pure functions

Advantages of Structured Plans and Operators

- **Auditability:** Every transformation is explicitly named and parameterized. There is no hidden logic inside a generated code block — the plan is the full record of what happened.
- **Determinism:** Operators are pure functions with no side effects. Running the same plan twice on the same data always produces the same result.
- **Separation of concerns:** The LLM does all the reasoning at planning time. The executor does no interpretation — it cannot make decisions. This means bugs are always either in the plan (LLM reasoning) or in an operator (well-isolated code), never entangled.
- **Execution traces:** The step-by-step row count trace is a natural by-product of the pipeline model and would require significant extra instrumentation in a code-generation agent.

Limitations vs. Code Generation

- **Expressiveness:** The operator set is finite. Questions that require joins, window functions, conditional aggregations, or string manipulation cannot be expressed without adding new operators. A code-generation agent can express anything Python can.
- **Multi-table queries:** HW3 only operates on a single table. Joining two datasets requires explicit operator support; a code-generation agent handles this for free.
- **Operator gaps cause silent failures:** As shown in Failure Case 2, a missing OR/IN capability caused the executor to return 0.0 with no error. A code-generation agent would have written `df[df['make'].isin(...)]` correctly by default.

Trade-offs

Trade-off	Discussion
Flexibility vs. Reliability	Code generation is maximally flexible but unreliable — the LLM can hallucinate column names, use deprecated APIs, or produce code with off-by-one errors. Structured plans are less flexible but every step is validated against a known operator signature before execution.

Trade-off	Discussion
Expressiveness vs. Controllability	Arbitrary code can express any computation; structured operators can only express what has been explicitly implemented. However, operators are easy to test in isolation and their behavior is fully predictable, which makes the system much easier to control and extend safely.
Automation vs. Interpretability	A code-generation agent requires less design effort up front — just prompt the LLM. A plan-and-execute system requires designing the operator vocabulary, the plan schema, and the prompt that bridges them. In return, every execution is fully interpretable: the plan explains the reasoning and the trace explains the data transformations.

7. Reflection

Easy vs. difficult question types: The system handles aggregate-rank-truncate patterns (group → sort → limit) extremely reliably — these are the most natural fit for the operator model. Questions with a single filter dimension followed by aggregation also work well. The system struggles with questions that require OR logic across the same column (e.g., "BMW or Mercedes"), questions comparing a row's value to a computed aggregate (e.g., "above average"), and any question involving two tables.

Limitations of the current operator set: - No OR logic within a single `filter_rows` step — the `in` operator was added as a patch but a general `logic: "or"` field would be cleaner. - No window functions — rank-within-group, running totals, and lag/lead comparisons are not expressible. - No string operators — filtering by substring (e.g., "violations containing PARKING") requires an exact match. - No join operator — cross-dataset questions are impossible. - `derive_columns` only supports arithmetic — no conditional derivations (CASE WHEN equivalent).

Additional operators that would improve the system: - `filter_rows` with `logic: "or"` field to support disjunctions - `join_tables` to merge two DataFrames on a key - `pivot` and `unpivot` for reshaping - `derive_columns` with a `conditional` type for if/else expressions - `rename_columns` for cleaner output column naming

Future extensions: The plan-and-execute model is well-suited for adding a **plan verification step** between the planner and executor — a second LLM call that checks the plan for logical errors before execution. This would catch silent failures like the OR-logic bug before they produce wrong answers. The system could also be extended to support **multi-step re-planning**: if the executor hits an error, it sends the error and partial trace back to the planner to generate a corrected plan, making the system self-healing.